

From One Photo to a Printable Object: A Validated Image-to-3D Pipeline on Apple Silicon

Student One, Student Two, Student Three,
Student Four, Student Five, Student Six,
Mentor Name

student.one@kaust.edu.sa, student.two@kaust.edu.sa,
student.three@kaust.edu.sa, mentor@kaust.edu.sa

July 16, 2026

Abstract

We present a single-image-to-3D-print pipeline that runs entirely on Apple Silicon and, unlike existing image-to-3D tools, *measures* what it does to the geometry it repairs. Generative image-to-3D models such as TRELLIS.2 produce meshes that look correct in a viewer but are not manufacturable: our test asset, generated from one photograph, contains **2.2M triangles spread over 4,518 disconnected components** and is not watertight, so a slicer cannot print it. Repairing such a mesh is inherently destructive — the fix that guarantees watertightness (voxel remeshing) also silently reshapes the object — and no existing tool reports how much shape was lost. Our system has three parts: (i) a port of the 4B-parameter TRELLIS.2 model from CUDA to PyTorch MPS/Metal, generating a textured mesh from one image in about five minutes on an M4 Pro; (ii) a print-preparation stage that discards floating-point noise, guarantees watertightness, and flood-fills the interior solid; and (iii) a **validation layer** — the core contribution — that quantifies the damage using topology-agnostic symmetric Chamfer and Hausdorff surface distances, volume change, and printability diagnostics. Because repair changes vertex count by over an order of magnitude, correspondence-based metrics are unusable; our point-to-surface formulation requires only that the two meshes occupy the same space. The validation layer turns an unverifiable black box into a measured one: it reports exactly **0.00% deviation** when post-processing is a no-op, and on the real asset it quantifies the accuracy/manufacturability trade-off that the user is being asked to accept.

Date: July 16, 2026

Code: <https://github.com/shivampkumar/trellis-mac>

Contents

1	Introduction	3
2	Related Work	4
3	Methodology	4
3.1	System overview	4
3.2	Stage 1: Generation on Apple Silicon	4
3.3	Stage 2: Print preparation	5
3.4	Stage 3: The validation layer	5
3.4.1	Why not compare vertices?	6
3.4.2	Fidelity metrics	6

3.4.3	Printability diagnostics	6
3.5	Evaluation Procedure	7
4	Results	7
5	Discussion	8
6	Limitations and Future Work	9
7	Conclusion	10

1 Introduction

Motivation. Digital fabrication labs receive a recurring request: “*I have a photo of this object — can you print me one?*” Until recently the answer required a trained user and hours of manual CAD work. Generative image-to-3D models have made the first half of that request nearly trivial: TRELLIS.2 [1] reconstructs a textured 3D asset from a single photograph in minutes. The promise is significant for teaching labs, museums, and rapid prototyping, where the bottleneck is modelling skill rather than printer access.

Problem statement. The promise does not survive contact with a 3D printer, for two reasons.

1. **Hardware access.** TRELLIS.2 and its dependencies are written for CUDA. A fabrication lab equipped with Apple Silicon machines cannot run it at all.
2. **Generated meshes are rendering assets, not manufacturing assets.** A generative model is trained to look correct from the outside. Nothing in its objective requires the mesh to be watertight, manifold, or free of disconnected debris — and in practice it is none of these. Our test asset is a single photographed object, but the generator emits it as **4,518 disconnected components**, not watertight, with a hollow interior. A slicer given this file either refuses it or silently produces a nonsense toolpath.

Repairing the mesh is possible, but the repair is where the real difficulty lies. The only reliable way to *guarantee* watertightness is to throw the original topology away and rebuild the surface from a voxel grid — an operation that reshapes the object. The user is therefore asked to accept a trade: manufacturability in exchange for fidelity. Existing tools ask the user to accept that trade *blind*. They report success or failure, never the magnitude of the distortion they introduced.

Key assumptions and scope. We assume a single, roughly object-centric input photograph and consumer fused-deposition printing. We assume the generated mesh, however malformed, is geometrically *correct enough* — our validation layer measures deviation introduced by *post-processing*, not the accuracy of the generation itself (Section 6). We do not attempt to auto-correct thin walls or overhangs, because both require shape-altering edits that we consider unsafe to apply without user consent.

Proposed system. We contribute a three-stage pipeline:

- **Generation.** A port of the TRELLIS.2 4B flow-matching model from CUDA to PyTorch MPS with Metal backends, producing a textured mesh from one image in ≈ 5 minutes on an M4 Pro.
- **Print preparation.** Noise-component filtering, best-effort repair with a guaranteed voxel-remesh fallback, exterior flood-fill to make the object solid, appearance re-baking, and decimation.
- **A validation layer.** A measurement stage that reports, for every run, how far the printable mesh has moved from the mesh the generator actually produced, together with printability diagnostics.

The validation layer is the contribution we consider most important, and it is the focus of this report. It is what converts an opaque, irreversible repair into an auditable one.

2 Related Work

Generative image-to-3D. TRELLIS [2] introduced Structured LATents (SLat), coupling a sparse voxel grid with dense multi-view features from a vision foundation model, decodable to radiance fields, Gaussians, or meshes. TRELLIS.2 [1] replaces SLat with *O-Voxel*, a field-free sparse voxel structure that encodes geometry and PBR appearance jointly and — key for our use case — supports arbitrary topology including open, non-manifold, and fully-enclosed surfaces. We adopt TRELLIS.2 as our generator because it is the strongest available single-image model with native PBR output. Its conditioning stack supplies our other dependencies: DINOv3 [3] for image features and RMBG-2.0, built on BiRefNet [4], for background removal.

Crucially, none of these works target manufacturability. TRELLIS.2’s ability to represent non-manifold and open surfaces is a *feature* for rendering and precisely the *problem* for printing: the representation is free to emit geometry a printer cannot interpret. This is the gap our work addresses.

Direct mesh generation. An alternative to repairing dense output is generating clean topology in the first place. TreeMeshGPT [5] autoregressively produces compact, artist-like meshes with orders of magnitude fewer faces. We evaluated it as a replacement for the TRELLIS.2 decoder and did not adopt it: it generates untextured geometry from a point cloud rather than PBR assets from an image, and its output is optimised for editability, not watertightness — so it would not remove our need for a repair-and-validate stage. We report it here because the comparison motivates our design: no current generator outputs print-ready topology, so *some* validated repair stage is unavoidable.

Mesh repair and measurement. Our repair path rests on classical geometry: marching cubes [6] to rebuild a surface from the filled voxel grid, and quadric error metric decimation [7], via `fast_simplification` [8], to bring the inflated face count back down. Our fidelity metric follows the standard symmetric point-to-surface formulation of Aspert et al. [9], whose central argument — that mesh error must be measured surface-to-surface rather than vertex-to-vertex when topology differs — is exactly our situation. Implementation uses trimesh [10] and xatlas [11]. The gap we fill is one of integration rather than algorithms: these error metrics are standard in mesh-processing research but are absent from generative image-to-3D tooling, which reports repair as a boolean rather than a measurement.

3 Methodology

3.1 System overview

The pipeline runs in three stages, each a standalone command:

```
# Stage 1: photo -> textured mesh (TRELLIS.2 on MPS)
python generate.py photo.png

# Stages 2 and 3: mesh -> printable mesh, validated
python make_printable.py output_3d.glb
```

Listing 1: The pipeline end to end.

3.2 Stage 1: Generation on Apple Silicon

Porting a 4B-parameter CUDA pipeline to MPS is largely a matter of surviving the places where the Metal stack behaves unlike CUDA. Three were significant:

- **Backend selection with fallbacks.** Sparse convolution runs on a Metal `flex_gemm` backend where available, falling back to a pure-PyTorch path when the `metallib` fails to load (it ships MSL 4.0, which only loads on macOS 26+). Attention uses SDPA. `PYTORCH_ENABLE_MPS_FALLBACK` must be set before torch is imported anywhere, or unsupported ops crash rather than falling back to CPU.
- **The macOS GPU watchdog.** Long-running Metal kernels in the SLat decoder are killed by the OS watchdog. The failure is pernicious: it does *not* raise a Python exception. Execution continues with empty tensors and fails later with an unrelated error. We detect the known downstream signatures and report the real cause with actionable workarounds instead of a misleading stack trace.
- **Texture baking.** A Metal-accelerated baker is used when available, with a pure-Python KDTree baker (UV-unwrapped via xatlas [11]) as fallback.

3.3 Stage 2: Print preparation

Given the generated `.glb`, the preparation stage applies:

1. **Component filtering.** The mesh is split into connected components and ranked by bounding-box diagonal. Components below `-min-object-ratio` (default 5%) of the largest are discarded as floating-dot noise. This runs in every mode, since generated assets carry thousands of specks that a slicer would treat as real objects.
2. **Best-effort repair.** Vertex merging, degenerate-face removal, hole filling, and normal fixing. If this achieves watertightness and solid infill is not requested, the pipeline stops here — this is the *fidelity-preserving* path.
3. **Guaranteed repair (voxel remesh).** Otherwise the mesh is voxelised and the interior flood-filled, then a surface is rebuilt with marching cubes [6]. Two details matter:
 - Filling uses the `orthographic` method, not the default hole-filling method. The latter fills only fully-enclosed voids, so any pinhole in the voxel shell lets the fill leak and the object stays a hollow shell — which appears in the slicer as floating islands. The orthographic method fills every voxel between the first and last surface voxel along each axis, producing one solid mass even when the shell leaks.
 - Voxelisation uses `max_iter=None` so the subdivision depth is computed from the mesh’s longest edge. The library default caps at 10 passes and simply fails on fine pitches.
4. **Appearance re-baking.** The voxel path destroys the original topology and UV atlas, so colour is resampled onto the new vertices by closest-point and barycentric interpolation from the original surface.
5. **Decimation.** Marching cubes inflates the face count, so the mesh is decimated back toward a target using QEM [7]. The collapse map returned by `fast_simplification` is replayed to keep per-vertex colours consistent with the decimated vertices.

Objects retain their original orientation. An earlier version auto-oriented each object onto a “best” resting pose by convex-hull search; we removed it because it silently moved the user’s model and the choice of pose is better left to the slicer.

3.4 Stage 3: The validation layer

Every step in Section 3.3 is lossy, irreversible, and invisible. The validation layer exists to make the loss visible. It has two halves: a *fidelity report* that measures deviation from the generator’s output, and a *printability report* that flags manufacturing risks.

3.4.1 Why not compare vertices?

The obvious way to compare a mesh before and after repair — match vertices and measure displacement — is unusable here, because repair does not preserve correspondence. It does not even preserve *count*: in our measurements the voxel path takes the 1.1M-vertex, 2.23M-face input, rebuilds an entirely new surface, and decimates it to a different mesh at 45.5% of the original face count (Table 2). Every vertex is new; there is no vertex i in the output corresponding to vertex i in the input.

We therefore measure **surface to surface**, following [9]. We sample $n = 20,000$ points uniformly by area from each mesh and compute point-to-surface distances in both directions. This requires only that both meshes occupy the same coordinate frame — which holds because our pipeline never moves the object — and is completely agnostic to vertex count, ordering, and topology.

3.4.2 Fidelity metrics

Let A be the pre-repair mesh, B the processed mesh, and S_A, S_B the sampled point sets. With $d(p, M)$ the distance from point p to the closest point on the surface of mesh M , we report:

$$d_{\text{chamfer}} = \frac{1}{2} \left(\frac{1}{|S_A|} \sum_{p \in S_A} d(p, B) + \frac{1}{|S_B|} \sum_{q \in S_B} d(q, A) \right) \quad (1)$$

$$d_{\text{hausdorff}} = \max \left(\max_{p \in S_A} d(p, B), \max_{q \in S_B} d(q, A) \right) \quad (2)$$

Both are reported in absolute units *and* as a percentage of the model’s bounding-box diagonal, which makes them scale-free and comparable across objects of different sizes.

The two metrics are deliberately complementary, and reporting only one would be misleading:

- **Chamfer** (Eq. 1) is a mean, so it describes typical deviation — but it *averages away* localised damage. A repair that is perfect everywhere except that it amputates one thin antenna still reports an excellent Chamfer score.
- **Hausdorff** (Eq. 2) is a maximum, so it catches exactly that amputation. It is correspondingly sensitive to single outliers.

Read together, a low Chamfer with a high Hausdorff is the signature of *localised* damage — the case that matters most, because it is the case a user would notice on the printed object and that a mean-only metric hides.

Two further quantities are reported:

- **Volume change (%)**, reported *only* when both meshes are watertight. Volume is undefined for a non-watertight mesh, and since the input to repair is typically not watertight, this field is usually N/A. Guarding it rather than printing a meaningless number is deliberate: a validation layer that emits confident nonsense is worse than one that admits ignorance.
- **Face count ratio (%)**, which exposes the topological cost of the repair.

3.4.3 Printability diagnostics

The second half of the layer flags manufacturing risks, and *reports rather than fixes* them:

- **Overhang area.** The fraction of total surface area whose face normal points downward beyond the 45° rule, i.e. faces with $n_z < -\cos(45^\circ)$, weighted by face area. Predicts support material requirements.

- **Thin walls.** 500 rays are cast inward from randomly sampled face centres along the inward normal; the first-hit distance approximates local wall thickness. Distances below 1.0 mm ($\approx 2.5\times$ a typical 0.4 mm nozzle) are counted as warnings.

Neither is auto-corrected. Thickening a wall or adding a chamfer changes the object’s visible shape, and we judged silent shape edits a worse failure than an informed warning — consistent with the reasoning that removed auto-orientation.

3.5 Evaluation Procedure

Tools. PyTorch (MPS backend), TRELLIS.2-4B [1], trimesh 4.12.2 [10], scikit-image (marching cubes), `fast_simplification` [8], xatlas [11]. Hardware: Apple M4 Pro, 24 GB unified memory.

Test input. We evaluate on a real asset produced by Stage 1 from a single photograph — not a synthetic or hand-authored mesh, since the pathologies we target are precisely those a generator introduces. Its properties are given in Table 1.

Baseline. Defining a baseline requires care, because the naive comparison (“mesh before vs. after”) is what our metric already computes. We therefore compare the two *policies* the tool can apply, which represent the two genuine options a user has:

- **Baseline — best-effort repair** (`-no-solid-infill`): conventional cleanup, preserves topology, but offers *no watertightness guarantee*.
- **Full system — guaranteed solid** (default): voxel remesh with exterior flood fill; always watertight and solid.

Metrics. Watertightness (the binary gate that decides whether the object is printable at all), Chamfer and Hausdorff deviation, volume change, face ratio, overhang area, thin-wall warnings, and wall-clock time.

Correctness check on the metric itself. A validation layer must itself be validated. We verify that when post-processing is a genuine no-op (an already-watertight mesh through the best-effort path), the fidelity report returns exactly zero on every metric — confirming that non-zero readings reflect real geometric change and not sampling noise in the measurement.

4 Results

Table 1: The generated test asset, as emitted by TRELLIS.2 from one photograph. This is the input to print preparation.

Property	Value
Vertices	1,113,494
Faces	2,229,064
Watertight	No
Connected components	4,518
Components kept as real geometry	1

Table 1 is itself a result, and the headline one: a photograph of *one* object becomes a mesh of **4,518 disconnected pieces**, of which exactly one is the object and 4,517 are noise. This is the

concrete form of the gap identified in Section 1 — the generator’s output is visually convincing and structurally unusable.

Table 2: Full print-preparation run on the asset in Table 1, default (guaranteed-solid) policy. Every number is measured by the validation layer on a single run; timing is wall-clock on an M4 Pro.

Quantity	Input	Processed
Watertight (printable at all)	No	Yes
Faces	2,229,064	≈1,000,000
Face count vs. input surface	100%	45.5%
<i>Fidelity of processed vs. input surface (Eqs. 1, 2)</i>		
Chamfer distance (mean deviation)	—	0.00100 (0.09%)
Hausdorff distance (worst deviation)	—	0.05033 (4.68%)
Volume change	—	N/A (input not watertight)
<i>Printability diagnostics</i>		
Overhang area (> 45°)	—	31.2% of surface
Thin-wall warnings (< 1 mm, 500 rays)	—	499
Repair time	—	78 s
Total wall-clock	—	104 s

The default and `-no-solid-infill` policies produced an *identical* result on this asset (both 87s repair, same output mesh), because best-effort cleanup alone could not make the mesh watertight — so both fell through to the voxel path. This is itself a finding (Section 5): on real generative output the “preserve topology” baseline is not actually available, and the guaranteed path is not optional.

Table 3: Validation-layer self-check: an already-watertight mesh through the best-effort path. Exact zeros confirm the metric reports real geometric change rather than sampling noise.

Metric	Value
Chamfer distance	0.00000 (0.00%)
Hausdorff distance	0.00000 (0.00%)
Volume change	+0.00%
Face count ratio	100.0%

5 Discussion

The two metrics disagree, and that is the result. The single most informative number in Table 2 is not either distance on its own but their *ratio*. The Chamfer distance is 0.09% of the model size — by any reasonable standard, an excellent repair: averaged over the whole surface, the printable mesh sits a tenth of a percent away from what the generator produced. Read alone, it would justify shipping the result without a second thought. The Hausdorff distance is 4.68%, roughly **50 times larger**. This is exactly the localised-damage signature predicted in Section 3.4: the repair is faithful almost everywhere and badly wrong in a small region — most likely a thin protrusion that the voxel grid could not resolve and marching cubes rounded off. A mean-only report would have hidden precisely the defect a user would notice on the printed object. This is the concrete payoff of reporting both metrics, and it is why we consider the validation layer, not the port, the contribution.

The guards fire correctly. Two fields report N/A or a self-check zero rather than a number, and both are correct to do so. Volume change is N/A because the input mesh is not watertight, so its volume is genuinely undefined — printing a plausible-looking percentage there would be fabrication. And on the identity self-check (Table 3) every metric reads exactly zero, confirming that the 0.09%/4.68% readings above are real geometric change and not an artifact of area sampling. A measurement tool earns trust by being verifiably silent when there is nothing to report; ours is.

An unexpected finding: the baseline collapsed. We designed the evaluation around two policies, expecting the best-effort path to preserve topology at some fidelity cost and the solid path to guarantee printability at a larger one. On real generative output the distinction vanished: best-effort repair could not close the mesh, so both policies took the voxel path and returned the same result. The lesson is that conventional mesh cleanup, which assumes near-manifold input, is simply the wrong tool for the output of an image-to-3D model — the geometry is too far from watertight for hole-filling to recover. This validates the decision to make the aggressive voxel path the default rather than an opt-in.

Where the diagnostics meet, and miss, expectations. The overhang figure (31.2%) is plausible for an organic, curved object and is the kind of number a user can act on by adding supports. The thin-wall count (499 of 500 rays) is the opposite — it is almost certainly an artifact, not a measurement, because the mesh is in normalised coordinates where the fixed 1 mm threshold flags essentially the entire surface. We report it honestly as a limitation (Section 6) rather than dressing it up: a diagnostic that fires on everything conveys no information, and the fix is a physical scale, not a different threshold.

6 Limitations and Future Work

- **We validate the repair, not the reconstruction.** This is the most important limitation of the work. Our metrics measure deviation between the processed mesh and *the mesh the generator produced* — not between the processed mesh and the real photographed object. A hallucinated feature that TRELLIS.2 invents is faithfully preserved and reported as 0% deviation. The pipeline therefore certifies “*we did not damage what the model gave us,*” which is strictly weaker than “*this is an accurate replica.*” *Next step:* photograph the printed object from known viewpoints and measure against the input image, or evaluate against a ground-truth scanned dataset to characterise generation error separately from repair error.
- **The thin-wall metric assumes millimetre units.** The 1.0 mm threshold is compared directly against mesh coordinates, so it is only meaningful if the model is scaled to millimetres. Generated assets are typically in an arbitrary normalised frame, which makes the reported warning count unreliable in the default pipeline. *Next step:* require an explicit physical scale (or target print size) and convert before thresholding; report the metric as N/A when no scale is known, following the same guard already used for volume change.
- **Thin-wall sampling is sparse and stochastic.** 500 rays over a multi-hundred-thousand-face mesh is a thin sample, and it is unseeded, so counts vary between runs on identical input. *Next step:* seed the sampler for reproducibility and scale the ray budget with surface area; validate against a slicer’s own thickness analysis.
- **Single test object.** All quantitative results come from one generated asset. The numbers characterise a case, not a distribution. *Next step:* run the pipeline over a benchmark set spanning thin-featured, multi-part, and organic shapes, and report metric distributions rather than point values.

- **No end-to-end print verification.** We validate geometry, but we have not closed the loop by printing the validated meshes and confirming that the diagnostics predict real print outcomes. *Next step:* print a sample spanning the range of reported overhang and thin-wall scores and correlate the predictions with observed failures — the only way to establish that the thresholds are calibrated rather than plausible.
- **Hausdorff is unbounded by construction.** A single stray voxel speck surviving the noise filter dominates the worst-case metric. *Next step:* additionally report a robust quantile (e.g. 95th percentile) alongside the true maximum to separate systematic deviation from isolated outliers.

7 Conclusion

We built a single-image-to-3D-print pipeline that runs on Apple Silicon and, unlike existing image-to-3D tooling, measures the geometric cost of the repair it performs. A photograph becomes a textured mesh in about five minutes via a CUDA-to-MPS port of TRELLIS.2; that mesh — 2.2M triangles across 4,518 disconnected pieces, not watertight — is then made printable by a repair stage whose every step is lossy and irreversible. The core contribution is the validation layer that renders those losses visible: using topology-agnostic symmetric Chamfer and Hausdorff surface distances, it certified our test asset as average-faithful (0.09% mean deviation) while flagging a localised worst-case error 50 times larger (4.68%) that a mean-only metric would have concealed, and it reads exactly 0.00% when post-processing changes nothing — proving the measurement reports real change rather than noise. The takeaway is a shift in what an image-to-3D tool is accountable for: not merely “did it produce a printable file,” but “how far did making it printable move it from what the model generated, and where” — an answer the user can finally see before they commit the object to a printer.

References

- [1] Jianfeng Xiang et al. “Native and Compact Structured Latents for 3D Generation”. In: *arXiv preprint arXiv:2512.14692* (2025). TRELLIS.2; introduces the O-Voxel representation and the 4B flow-matching model used as this project’s generator. URL: <https://arxiv.org/abs/2512.14692>.
- [2] Jianfeng Xiang et al. “Structured 3D Latents for Scalable and Versatile 3D Generation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. Spotlight; introduces the SLat representation that TRELLIS.2 builds on. 2025. URL: <https://arxiv.org/abs/2412.01506>.
- [3] Oriane Siméoni et al. “DINOv3”. In: *arXiv preprint arXiv:2508.10104* (2025). Image feature extractor used to condition the TRELLIS.2 pipeline. URL: <https://arxiv.org/abs/2508.10104>.
- [4] Peng Zheng et al. “Bilateral Reference for High-Resolution Dichotomous Image Segmentation”. In: *CAAI Artificial Intelligence Research* (2024). Architecture behind RMBG-2.0, the background-removal stage of the pipeline.
- [5] Stefan Lionar, Jiabin Liang, and Gim Hee Lee. “TreeMeshGPT: Artistic Mesh Generation with Autoregressive Tree Sequencing”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025. URL: <https://arxiv.org/abs/2503.11629>.

- [6] William E. Lorensen and Harvey E. Cline. “Marching Cubes: A High Resolution 3D Surface Construction Algorithm”. In: *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. 1987, pp. 163–169. DOI: [10.1145/37401.37422](https://doi.org/10.1145/37401.37422).
- [7] Michael Garland and Paul S. Heckbert. “Surface Simplification Using Quadric Error Metrics”. In: *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. 1997, pp. 209–216. DOI: [10.1145/258734.258849](https://doi.org/10.1145/258734.258849).
- [8] Alex Kaszynski and Sebastien Sanchez. *fast_simplification: Python wrapper of Fast-Quadric-Mesh-Simplification*. Implementation of [7] used for decimation; its collapse replay provides the index mapping used to resample per-vertex attributes. 2024. URL: <https://github.com/pyvista/fast-simplification>.
- [9] Nicolas Aspert, Diego Santa-Cruz, and Touradj Ebrahimi. “MESH: Measuring Errors between Surfaces using the Hausdorff Distance”. In: *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)* (2002). Standard formulation of symmetric point-to-surface mesh error, the basis of this project’s fidelity metric, pp. 705–708. DOI: [10.1109/ICME.2002.1035879](https://doi.org/10.1109/ICME.2002.1035879).
- [10] Michael Dawson-Haggerty et al. *trimesh*. Version 4.12.2. Mesh I/O, repair, voxelisation, proximity queries and ray casting used throughout the print-prep and validation layers. 2025. URL: <https://github.com/mikedh/trimesh>.
- [11] Jonathan Young. *xatlas: Mesh parameterization / UV unwrapping library*. UV unwrapping for the fallback (KDTree) texture baker. 2022. URL: <https://github.com/jpcy/xatlas>.